

# Game Development in Javascript / Typescript

Curso 2025-2026

# ¿Who are we?



- Alejandro Feo Martín  
[alejandro.feo.24@ull.edu.es](mailto:alejandro.feo.24@ull.edu.es)
- Jose Angel Portillo Garcia  
[jose.portillo.11@ull.edu.es](mailto:jose.portillo.11@ull.edu.es)
- Kyliam Gabriel Chinaea Salcedo  
[kyl iam.chinea.35@ull.edu.es](mailto:kyl iam.chinea.35@ull.edu.es)



# The Phaser Framework

Link to [code examples](#) in these slides

# Topics

- What is Phaser?
- Create a project
- Main
- Scene
- GameObject
- Physics
- Comparison: Phaser vs Typescript Vanilla

# What is Phaser?

# What is it?

- **Fast and Open Source Framework for game development in web**



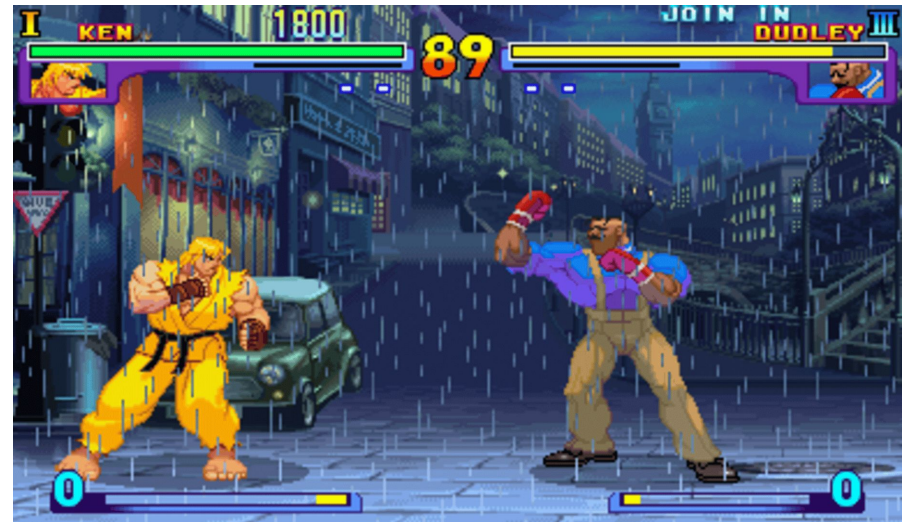
# What is it?

- **Multi Platform**



# When to use it?

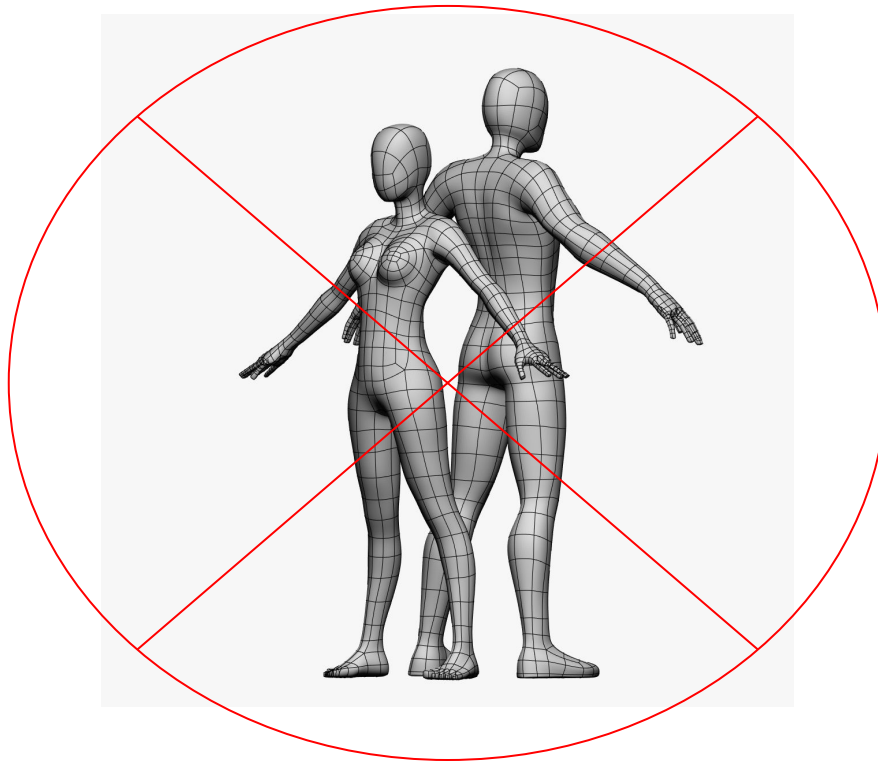
- 2D Games. Platforms, Visual Novels, Fighting Games





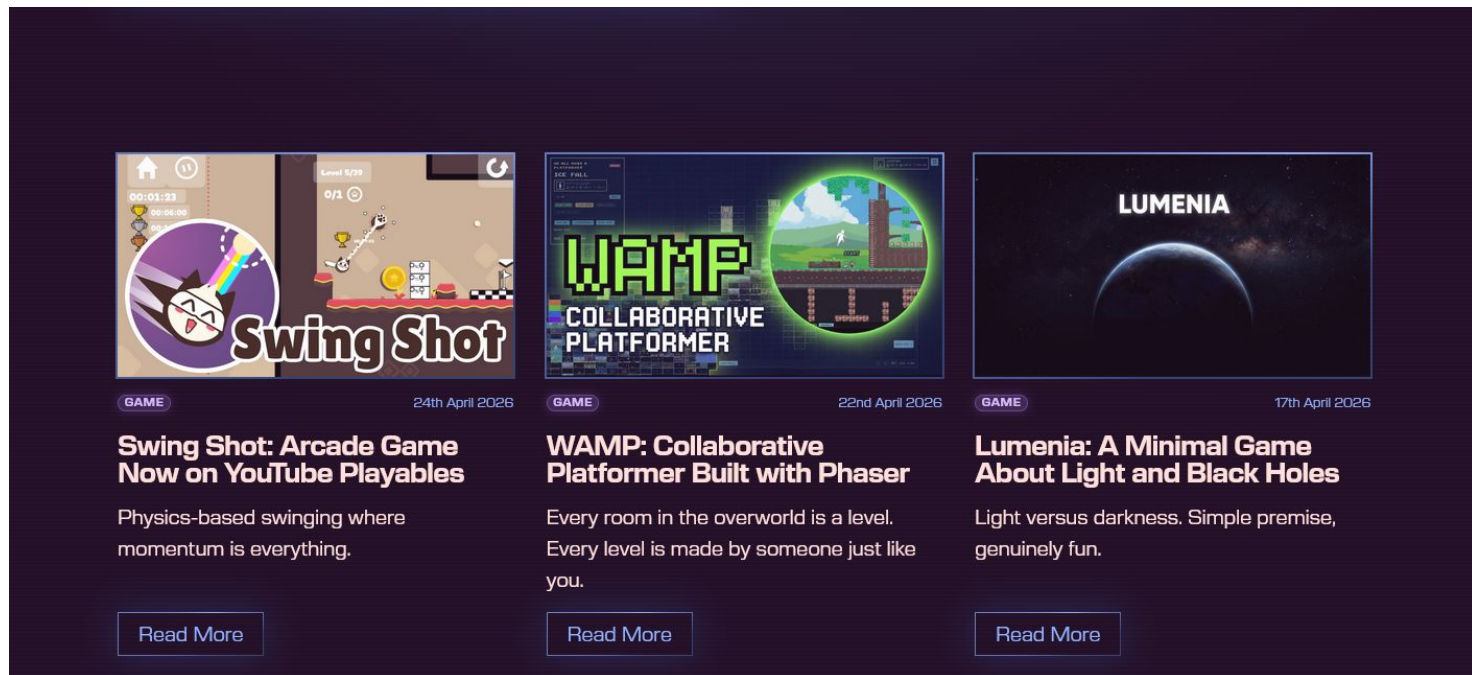
# When not to use it?

- **Any type of 3D Game**



# Want to check more?

- **Repository of games made in Phaser**
  - [List of games](#)



# How to install?

- **Installing**

- npm install phaser

- **Create Templates**

- npm create @phaserjs/game@latest

# Project Creation Demo

# Create a project

- **Instructions**

- `npm create @phaserjs/game@latest [game name]`

- **Recommended parameters**

- Web Bundler
- Vite template
- Minimal (1 scene) / complete (multiple scenes)
- TypeScript language

# Main file of the project

- **Minimal layout**
  - Imports
    - Phaser import
    - Scene imports
  - Configuration
  - Game creation

# Main file example

- `startGame()` - configuration and game creation

```
/**
 * Initializes and starts the Phaser game instance.
 */
function startGame() {
  const config: Phaser.Types.Core.GameConfig = {
    type: Phaser.AUTO,
    width: 800,
    height: 600,
    parent: 'DinamicoEstaticoDemo-app',
    scene: [DinamicoEstaticoDemo],
    backgroundColor: ColorName.CYAN,
    physics: {
      default: 'arcade',
      arcade: {
        gravity: { y: 300, x: 0 },
        debug: true
      }
    }
  };
  new Phaser.Game(config);
}
```

# Main file of the project

- **Useful parameters**

- type

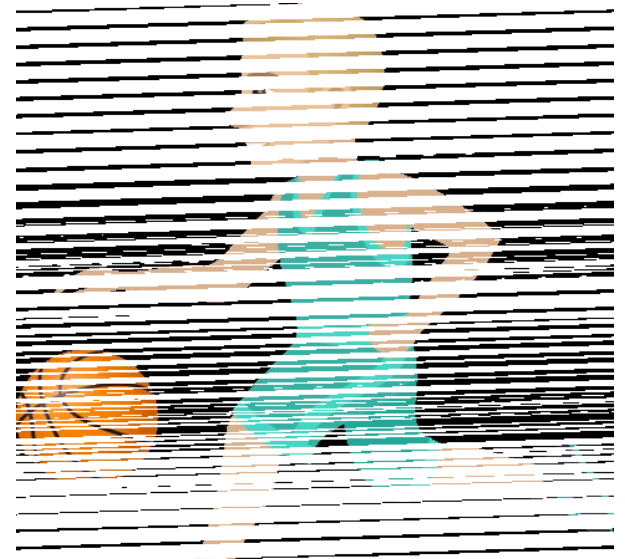
- Phaser.AUTO
    - Phaser.WEBGL
    - Phaser.CANVAS





# Main file of the project

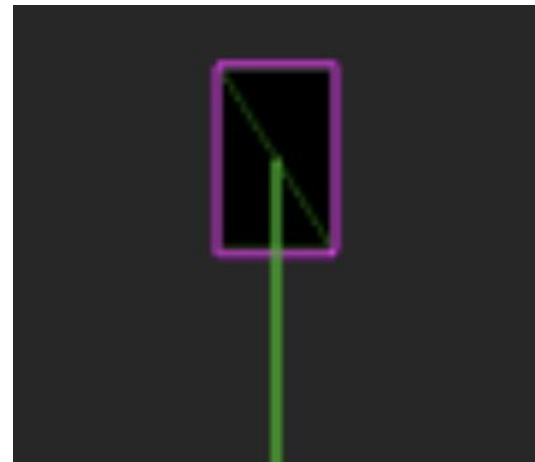
- **Useful parameters**
  - physics default engine
    - arcade
    - matter - more complex



# Main file of the project

- **Useful parameters:**

- debug
  - true
  - false



Shows hitboxes and vectors

# Main file of the project

- **Useful parameters:**

- `parent`

HTML element where the canvas will be injected

- `pixelArt`

To improve pixel art visualization

# Main file of the project

- **Phaser.Game( )**
  - Entry point of the application
  - Takes the config object and starts the engine
  - Automatically launches the first scene

# Scenes

# What is a Scene ?

Imagine that a game is not just a block of code, but a theatrical play.



# What is a Scene ?

## **Scene:**

It is the physical stage. It defines the set, the lighting, and which "act" we are performing (the main menu, level 1, Game Over).

# What is a Scene ?

## **GameObjects (Actors ):**

These are the characters and objects on stage. A Sprite is an actor in a costume; a Text is a narrator. They do not exist outside of the stage.

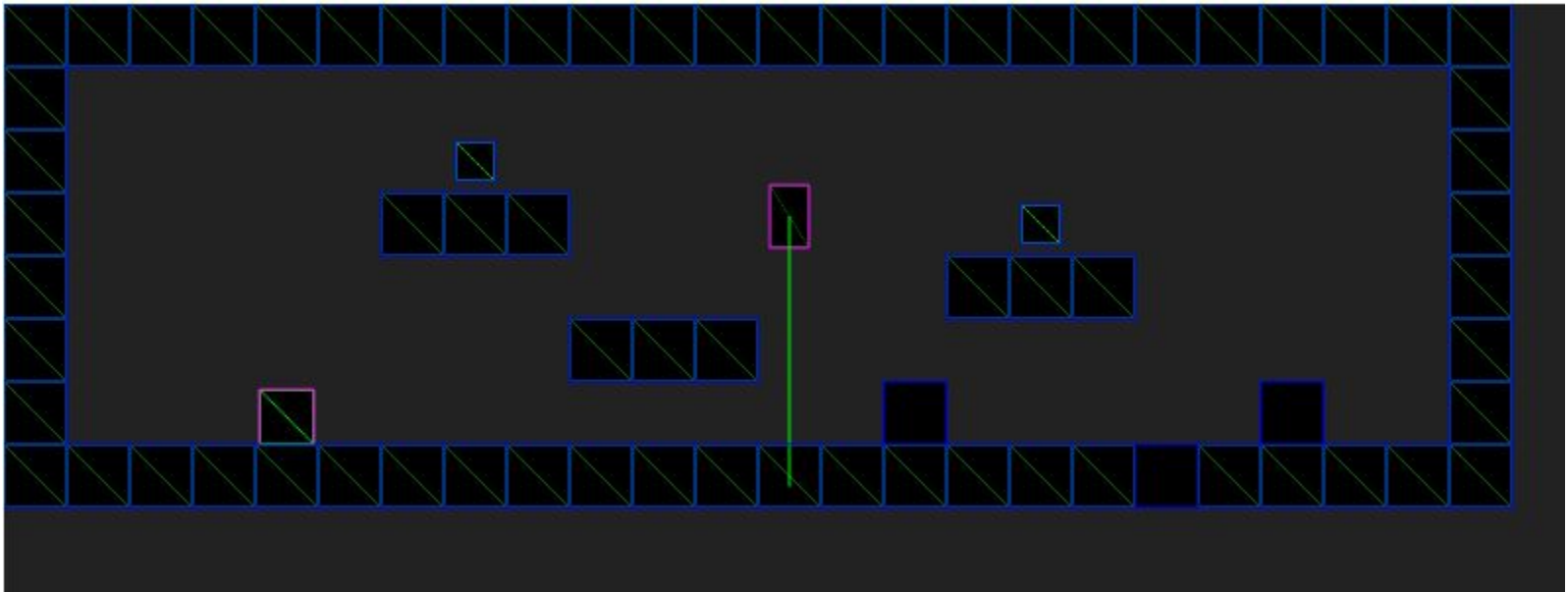


# What is a Scene ?

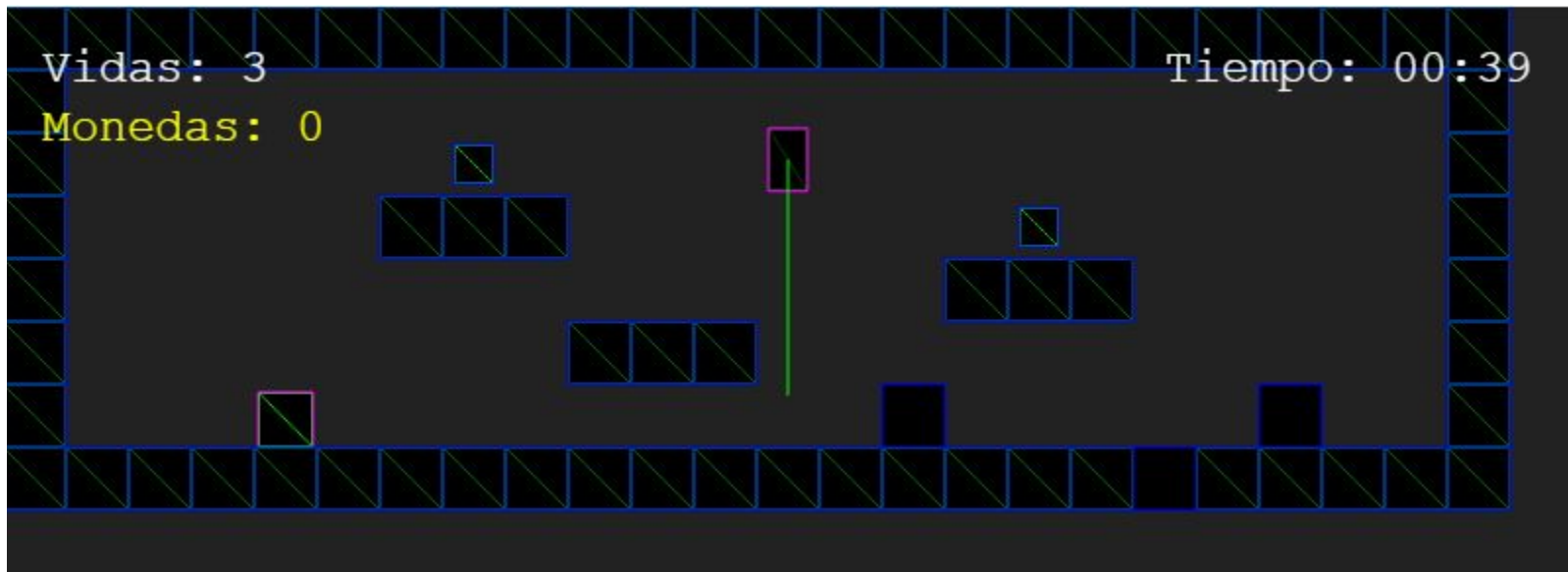
## **The Director (SceneManager):**

This is the one who decides when the curtain rises, when the lights go out, and which actors enter or leave.

# What is a Scene ?



# What is a Scene ?



# What is a Scene ?



# The Scene Lifecycle



# Parallel Scene Management

## **Launch:**

Starts a scene in parallel. Used for UIScene or Pause overlays while the game logic continues below.

## **Start:**

Shuts down the current scene and starts a new one. Used for level transitions, moving from the Main Menu to the Game.

# Parallel Scene Management

**Pause / Resume:** Halts update loop but keeps visibility.

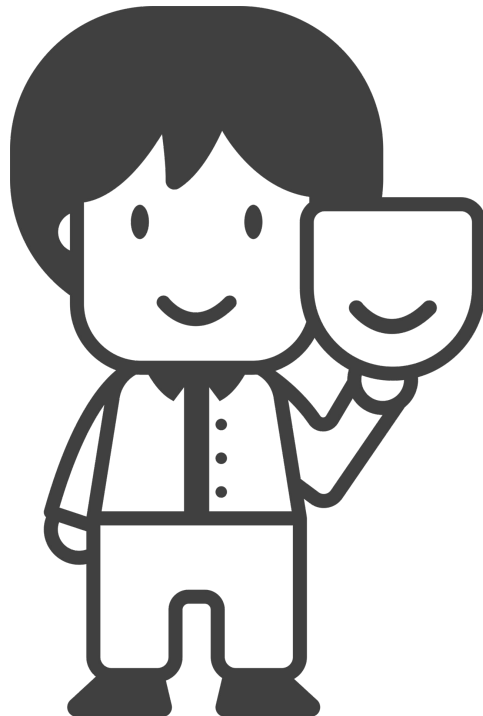
**Sleep / Wake:** Halts update AND hides the scene.

# GameObjects



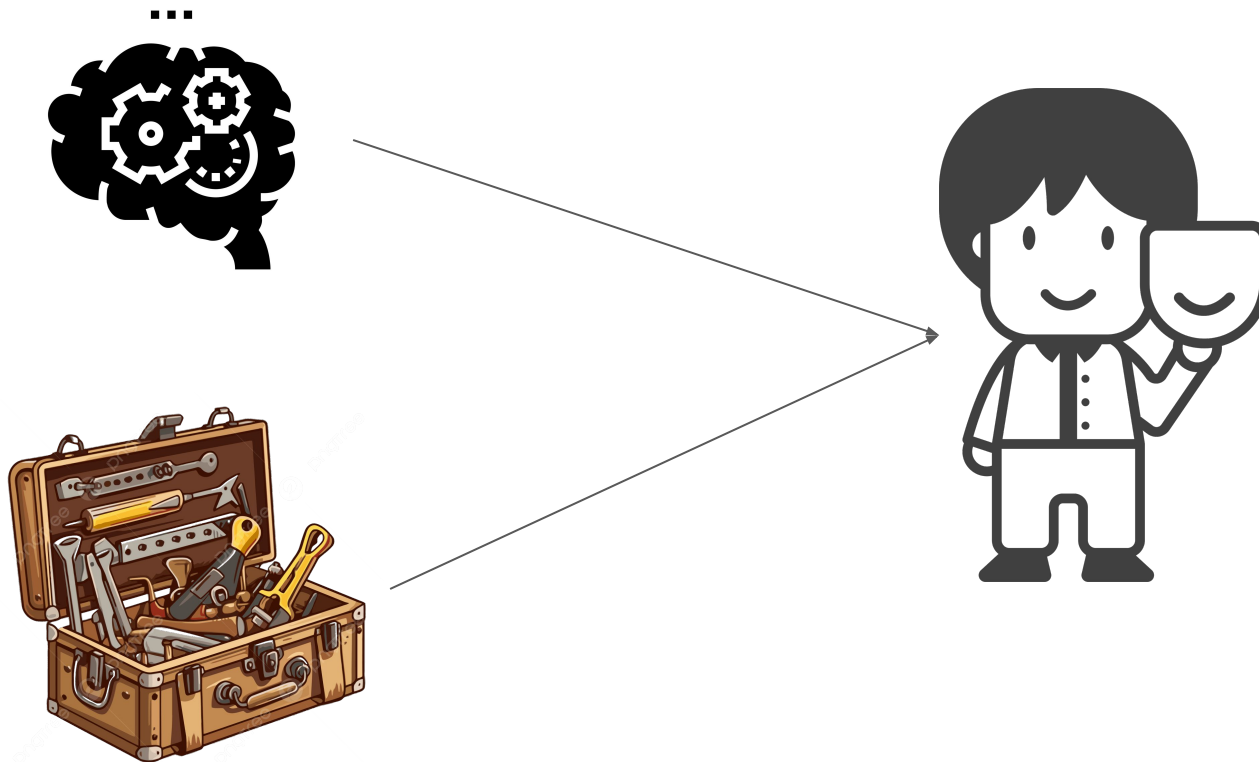
# What is a GameObject ?

- **If the game is the theatre then the different objects that participate are actors.**



# What is a GameObject ?

- **The Game Objects are the fundamentals of Scenes in Phaser**

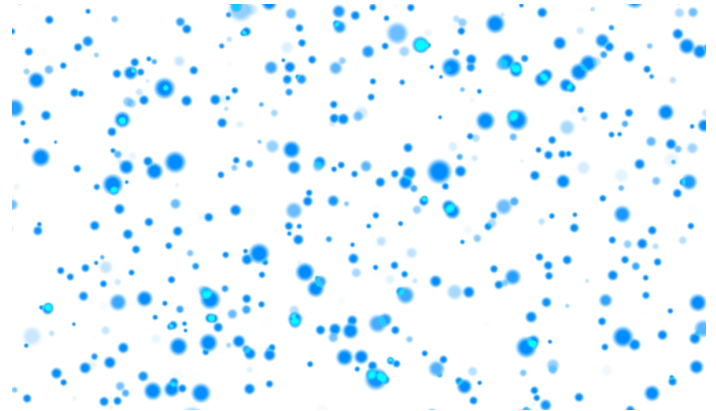
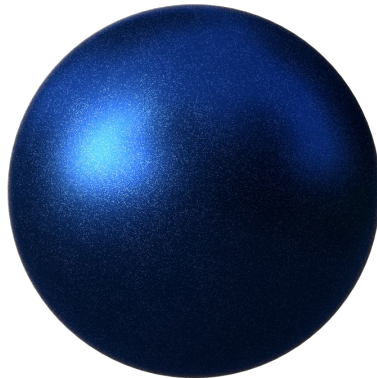
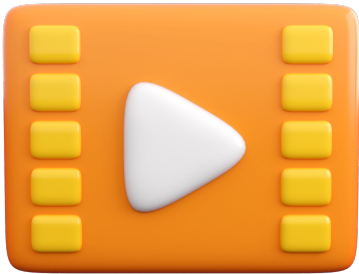


# What is a GameObject?

- **GameObject Structure**
  - `scene: Phaser.Scene`
  - `x: number`
  - `y: number`

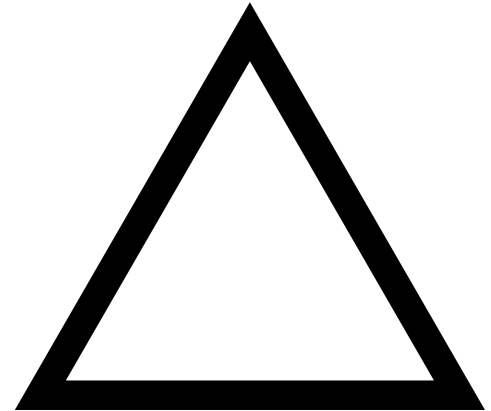
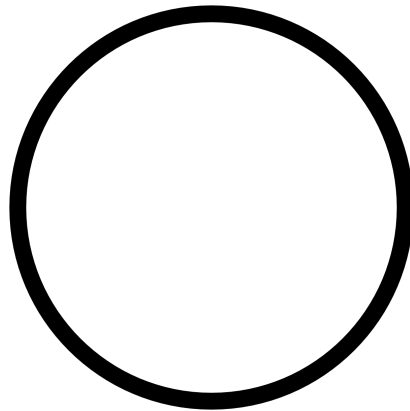
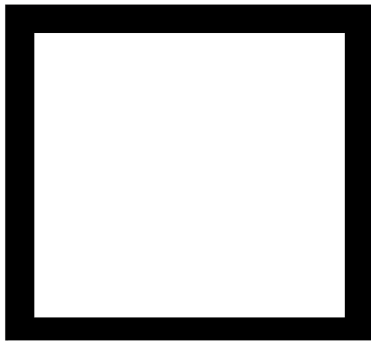
# What is a GameObject ?

- **There's numerous type of Game Objects**



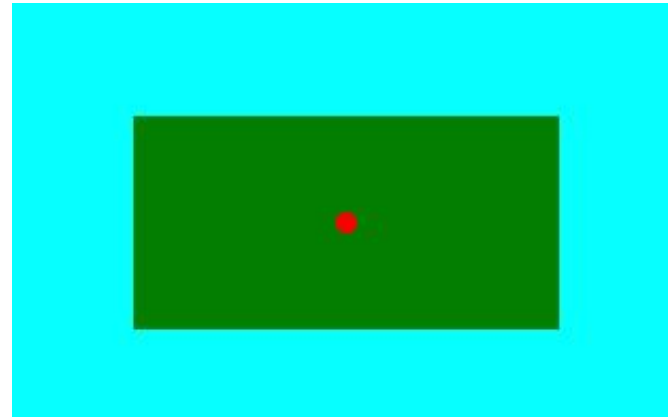
# Rectangle Game Object

- **The Game Object library accepts geometric figures**



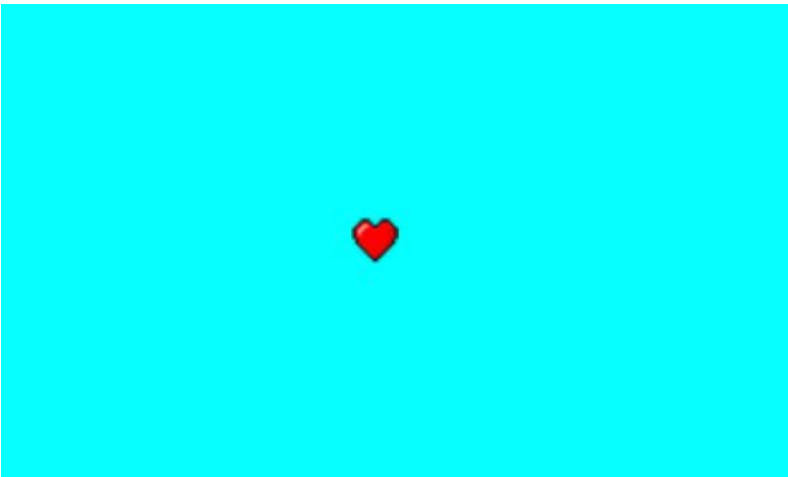
# Coordinate System

- **Game Objects can behave differently than elements in HTML**



# Image Game Object

- Images only need to be preloaded to be used



# Sprite Game Object

- First of all, what's an sprite?



Spritesheet



# Sprite Game Object

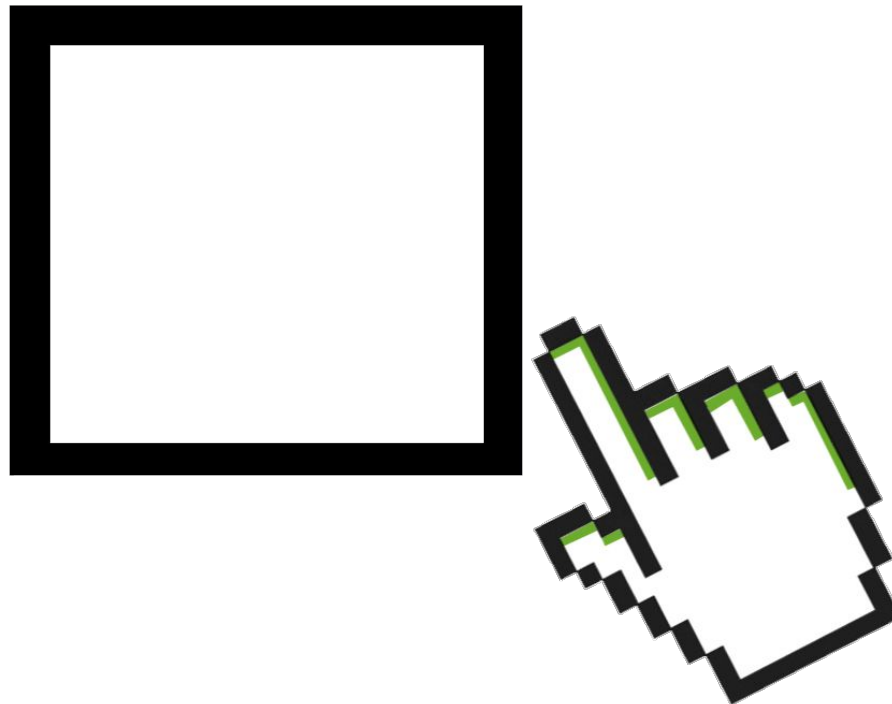
- Each frame counts as an state



**Spritesheet**

# Interactive Game Objects

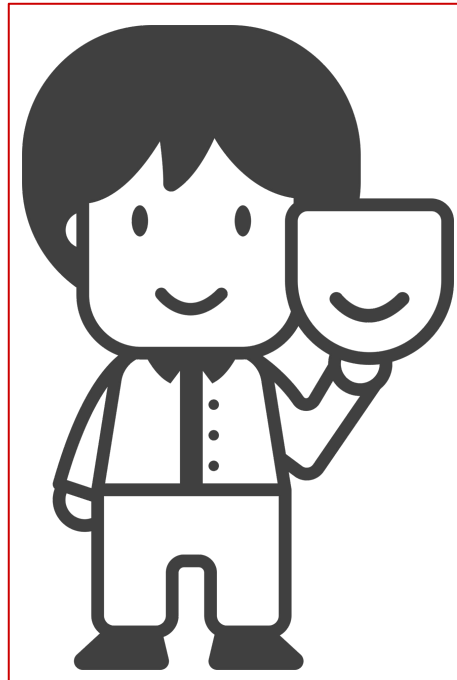
- You can make any kind of game Object be able to interact with the pointer



# How to implement it?

- **Pointer input**

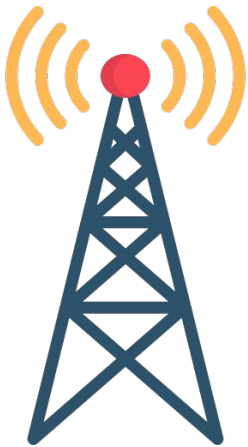
- `gameObject.setInteractive();`



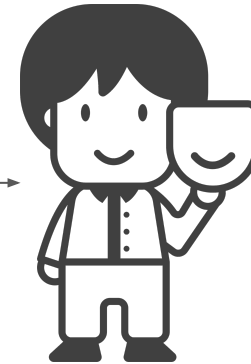
Hit Area

# Key inputs in Game Objects

- **Key inputs can modify the behaviour of the position of the Game Object**



'move-left'

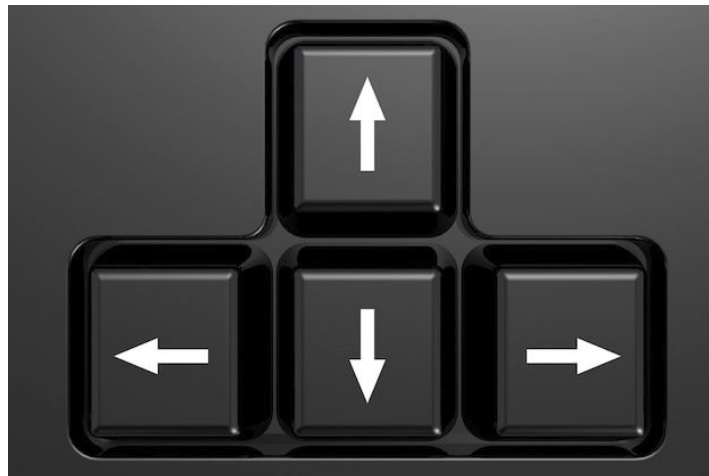


Key pressed



# How to implement it?

- **Keys inputs must be created before being used**
  - `this.cursors = scene.input.keyboard.createCursorKeys();`



# Physics

# Engine Choice: Arcade vs. Matter



VS



# Arcade Principles: Simplicity

## Supported Shapes

Arcade only calculates math for Rectangles and Circles.

Uses AABB (Axis-Aligned Bounding Boxes). Boxes cannot rotate mathematically during collision resolution, ensuring  $O(N)$  efficiency for hundreds of bodies.



# Dynamic vs. Static Bodies

## **Static: The Environment**

Fixed position. Removed from the acceleration/gravity integration loop. Perfect for platforms and walls.

```
// Wall.ts  
scene.physics.add.existing(this, true);  
// StaticBody: No gravity, no movement.
```

# Dynamic vs. Static Bodies

## Dynamic: The Hazards

Responsive to forces. Uses Euler integration for velocity and acceleration.

```
// LavaFalling.ts  
scene.physics.add.existing(this, false);  
body.setAllowGravity(true);  
body.setVelocityY(200);
```

# Collider vs. Overlap

| Feature             | Collider              | Overlap                |
|---------------------|-----------------------|------------------------|
| Detection           | Intersection of AABBs | Intersection of AABBs  |
| Physical Separation | Yes (Push-out)        | No (Ghosting)          |
| Callback Trigger    | On Contact            | On Intersection        |
| Typical Use Case    | Player vs. Floor      | Player vs. Coin / Lava |

# Essential Physics Methods

## Movement Control

### **setVelocity(x, y):**

Applies a constant velocity. It does not require manual updating in the game loop.

### **setBounce(x, y):**

Defines the coefficient of restitution (bounciness).

### **setCollideWorldBounds(bool):**

Activates the canvas boundaries as solid walls.

# Essential Physics Methods

## Synchronization and State

### **refreshBody():**

Synchronizes the collision box after changing the visual size (e.g., using `setDisplaySize`).

### **body.reset(x, y):**

Teleports the object to a new position, stopping all previous physical forces and momentum.

### **setEnabled(bool):**

Completely enables or disables the body within the physics engine.

# Phaser vs Vanilla Typescript

# Phaser vs Vanilla TypeScript

|  Area        |  Vanilla (src/) | Lines |  Phaser (game/) | Lines |  Key Change                                      |
|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-------|---------------------------------------------------------------------------------------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------|
|  Model       | model/                                                                                           | 919   | gameobjects/                                                                                      | 351   |  Core logic partially replaced by Phaser sprites |
|  View        | view/                                                                                            | 512   | scenes/                                                                                           | 465   |  Manual rendering → Phaser scene system          |
|  Controller  | controller/                                                                                      | 212   | GameScene + GameState                                                                             | 283   |  Logic redistributed into scenes and state       |
|  Root code | src/ root                                                                                        | 227   | game/ root                                                                                        | 180   |  Fewer custom utilities needed                 |
|  TOTAL     | Vanilla (src/)                                                                                   | 1870  | Phaser (game/)                                                                                    | ≈1163 |  <b>-38%</b><br>lines of code                  |

# References

- [Phaser documentation](#)
- [Phaser repository](#)

## Recommendation:

- [Build Your First Game with Phaser 4 - Playlist - Scott Westover](#)